ISE 573: Manufacturing Execution Systems

Week 5: Scheduling Optimization & Introduction to Metaheuristics

"When Exact Solutions Take Longer Than the Universe Has Existed"

Manbir Sodhi

Department of Industrial Engineering
University of Rhode Island

Spring 2026

Where Are We in the Course?

Our Journey So Far

- Week 1–2: MES Foundations, MESA-11
- Week 3: Math Modeling & Scheduling *Formulations*
- Week 4: Databases & SQL for MES
- ▶ **Week 5: Scheduling Optimization + Intro to Metaheuristics**
- Week 6: Lot Sizing, MRP & Production Planning

Week 3 Recap

We learned to *formulate* scheduling problems:

- Decision variables, objectives, constraints
- Single machine, parallel, flow shop, job shop
- Big-M disjunctive constraints

This week: How do we actually *solve* them?

Today's Roadmap

Part 1: The Scheduling Challenge

1. Why scheduling is hard (combinatorics)
2. Where we need blazingly fast decisions
3. Problem types with worked examples
4. Dispatching rules — simple but effective

Part 2: Metaheuristics

5. What are metaheuristics and why?
6. Neighborhood search concepts
7. Simulated Annealing (from metallurgy!)
8. Genetic Algorithms (from biology!)

Learning Goals

By the end of today you will be able to:

- Explain why exact solvers struggle at scale
- Apply dispatching rules by hand
- Describe how SA and GA work
- Solve a small SA problem step-by-step
- Explain the metallurgical annealing analogy

Prerequisites (Week 3)

Decision variables, objectives, constraints, LP, scheduling notation ($C_{\max}$, T_j, p_j, r_j, d_j)

Scheduling Is Everywhere

Manufacturing

A semiconductor fab processes 10,000+ wafers/month through 500+ steps on 200+ machines. Scheduling decisions happen every few minutes.

Logistics

UPS delivers 25 million packages daily. Each truck visits 100–200 stops. Routes are recalculated overnight — every night.

Healthcare

An operating room suite has 12 ORs, 40+ surgeries/day, each needing specific surgeons, nurses, and equipment. One delay cascades through the day.

Airlines

An airline schedules 5,000+ flights/day across 800+ aircraft and 30,000+ crew. A single disruption triggers massive rescheduling.

Common thread: Assign tasks to resources over time, subject to constraints, to optimize some objective.

When We Need Blazingly Fast Decisions

Some scheduling problems can't wait hours for an optimal answer.

High-Frequency Trading

Trades execute in **microseconds**. Order scheduling must happen in < 1 ms. A "good enough" answer *now* beats a perfect answer 1 second later.

Emergency 911 Dispatch

Dispatchers have **seconds** to assign the nearest ambulance. Wrong assignment = delayed response = lives at risk. No time for an LP solver.

Ride-Sharing (Uber/Lyft)

Millions of ride requests per hour. Each must be matched to a driver in **2–3 seconds**. A massive dynamic assignment problem solved in near-real-time.

Robotic Warehouse (Amazon)

Thousands of robots must be routed to pick items without colliding. Paths recalculated **every few seconds** as new orders arrive.

Key insight: In these settings, we need *heuristics* and *metaheuristics* — methods that find good solutions fast.

More Real-Time Scheduling Scenarios

5G Network Slicing

Base stations allocate bandwidth to thousands of users every **1 ms**. Each user has different quality-of-service needs. The scheduler runs at hardware speed.

Cloud Computing (AWS/Azure)

When you submit a job, a scheduler assigns it to one of millions of servers in **milliseconds**. It must balance load, minimize latency, and handle failures.

Power Grid Dispatch

Grid operators match electricity supply to demand in **real time**. Every 5 minutes, generators are dispatched. Get it wrong and blackouts follow.

Autonomous Vehicles

A self-driving car makes path-planning decisions every **100 ms**: which lane, what speed, when to brake. Scheduling maneuvers in a dynamic environment.

The pattern: As decision frequency increases, we trade optimality for speed. Dispatching rules and lightweight heuristics dominate.

The Combinatorial Explosion

How many possible schedules exist?

Single machine, n jobs: $n!$ possible orderings.

Jobs	Schedules	Comment
5	120	Easy
10	3.6 million	Manageable
15	1.3 trillion	Getting tough
20	2.4×10^{18}	More than grains of sand
25	1.6×10^{25}	Forget it

m machines, n jobs: roughly $(n!)^m$

For 10 jobs on 3 machines: $(3.6 \times 10^6)^3 \approx 4.7 \times 10^{19}$

The Hard Truth

Most scheduling problems are **NP-hard**:

- No known polynomial-time algorithm
- Checking all possibilities is not feasible
- Even Gurobi/CPLEX can struggle with 50+ jobs

What This Means for You

If checking one schedule takes 1 nanosecond, evaluating all schedules for 20 jobs takes $\approx$ **77 years**.

We need smarter strategies!

What Does NP-Hard Actually Mean?

Polynomial Time (P)

Problems solvable in time that grows as a polynomial of input size (n^2, n^3, etc.).

Examples:

- Sorting a list: $O(n \log n)$
- Shortest path in a graph: $O(n^2)$
- Linear programming: polynomial

These are "easy" — time grows manageably.

NP-Hard

No known algorithm solves these in polynomial time. Best known algorithms grow *exponentially*: 2^n, $n!$, etc.

Examples:

- Job shop scheduling
- Traveling salesman problem
- Most real scheduling problems

Doubling the input can *square* the runtime!

Practical Implication

Small problems ($n < 20$): Exact solvers work fine. **Medium** ($20 < n < 100$): Exact solvers may work with good formulations. **Large** ($n > 100$): Almost always need heuristics or metaheuristics.

Scheduling Problem Zoo (Refresher from Week 3)

Problem Type	**Setup**	**Typical Objective**	**Complexity**
Single Machine	1 machine, n jobs	Min $C_{\max}$ or $\sum T_j$	NP-hard (most)
Parallel Machines	m identical machines	Min $C_{\max}$	NP-hard
Flow Shop	All jobs same route	Min $C_{\max}$	NP-hard ($m \geq 3$)
Job Shop	Each job own route	Min $C_{\max}$	Strongly NP-hard
Seq.-Dep. Setups	Setup depends on order	Min total time	$\equiv$ TSP!

Week 3 Focus

Formulating these as math models (variables, objective, constraints).

This Week's Focus

Solving these when exact methods are too slow.

Example: The Pizza Shop Problem

You run a one-oven pizza shop. Five orders came in:

Order	Bake Time	Promised By	Arrived At
A	12 min	5:30 PM	5:00 PM
B	8 min	5:25 PM	5:02 PM
C	15 min	6:00 PM	5:05 PM
D	6 min	5:20 PM	5:03 PM
E	10 min	5:45 PM	5:08 PM

Question: In what order do you bake them to minimize total lateness?
This is a **single machine scheduling problem**!

Mapping to Week 3

- **Machine** = Oven
- **Jobs** = Pizza orders
- **Processing time** = Bake time
- **Due date** = Promise time
- **Ready time** = Arrival time
- **Objective** = Min $\sum T_j$

Total permutations

$5! = 120$ orderings.
Manageable here — but what if you had 20 orders?

Pizza Shop: Trying Different Sequences

Start time = 5:00 PM (minute 0). Due dates as offsets: $d_A = 30$, $d_B = 25$, $d_C = 60$, $d_D = 20$, $d_E = 45$.

Sequence: A–B–C–D–E (FCFS)

Job	Start	End	Due	Late
A	0	12	30	0
B	12	20	25	0
C	20	35	60	0
D	35	41	20	21
E	41	51	45	6

$\sum T_j = 27$ minutes total late

Sequence: D–B–A–E–C (EDD)

Job	Start	End	Due	Late
D	3	9	20	0
B	9	17	25	0
A	17	29	30	0
E	29	39	45	0
C	39	54	60	0

$\sum T_j = 0$ — perfect!

Earliest Due Date (EDD): Process jobs by due date. Minimizes maximum tardiness on a single machine. Simple, fast, often very good!

Dispatching Rules: Quick and Dirty Scheduling

Dispatching rules assign the next job based on a simple priority. No optimization — just pick and go.

Rule	Logic	Best For
FIFO	First in, first out	Fairness; easy to implement
SPT	Shortest processing time first	Minimizing avg. flow time and WIP
EDD	Earliest due date first	Minimizing max tardiness
WSPT	Weighted SPT (w_j/p_j desc.)	Minimizing weighted completion time
CR	Critical Ratio: $(d_j - t)/p_j$	Dynamic priority; adapts over time

Strengths

Lightning fast ($O(n \log n)$) • Easy to explain • Work well in dynamic environments

Limitations

No optimality guarantee • Myopic (ignore global picture) • Can't handle complex constraints

SPT Rule: A Warehouse Picking Example

Scenario: One packing station, six orders waiting:

Order	A	B	C	D	E	F
Pack time (min)	8	3	12	5	2	7

FIFO: A–B–C–D–E–F

Completions: 8, 11, 23, 28, 30, 37
Avg. flow time $= \frac{8+11+23+28+30+37}{6} = 22.8$ min
Avg. # orders in system $= \frac{137}{37} = 3.7$

SPT: E–B–D–F–A–C

Completions: 2, 5, 10, 17, 25, 37
Avg. flow time $= \frac{2+5+10+17+25+37}{6} = 16.0$ min
Avg. # orders in system $= \frac{96}{37} = 2.6$

SPT reduced average flow time by 30% and WIP by 30%. *Provably optimal* for minimizing mean flow time on a single machine.

Why Does SPT Work? An Intuition

Think about it this way:

Every job that's *waiting* contributes to WIP. If you process a short job first, you **release it quickly**, reducing the number of jobs in the system.

Analogy: At a supermarket, the "10 items or fewer" express lane exists for the same reason — let the quick customers through first so fewer people are waiting overall.

The Math

SPT minimizes $\sum C_j$ (total completion time), which also minimizes average flow time $\bar{F}$ and average WIP $\bar{N}$ (by Little's Law: $\bar{N} = \lambda \bar{F}$).

Numerical Proof

Two jobs: $p_1 = 2$, $p_2 = 8$.

Order [1, 2] (SPT):
$C_1 = 2$, $C_2 = 10$
$\sum C_j = 12$

Order [2, 1]:
$C_2 = 8$, $C_1 = 10$
$\sum C_j = 18$

Processing the short job first saves $18 - 12 = 6$ units of total waiting.

The short job "blocks" others for less time!

Parallel Machines: Call Center Example

Scenario: 3 identical agents. Eight calls queued:

Call	1	2	3	4	5	6	7	8
Duration	15	8	12	6	20	10	4	14

Total work = 89 min. Lower bound: $\lceil 89/3 \rceil = 30$ min.

LPT (Longest Processing Time First)

Sort descending: 5(20), 1(15), 8(14), 3(12), 6(10), 2(8), 4(6), 7(4)
Agent A: 5(20) + 4(6) = 26 min
Agent B: 1(15) + 6(10) + 7(4) = 29 min
Agent C: 8(14) + 3(12) = 26 min
Makespan = 29 min

Why LPT Works Well

- Assign big jobs first so they don't get "stuck" at the end
- Small jobs fill gaps naturally
- Guaranteed within $\frac{4}{3}$ of optimal
- Here: 29 vs. 30 lower bound — nearly optimal!

Johnson's Rule: 2-Machine Flow Shop

Scenario: Print shop — **Printing** then **Binding**. Four books:

Book	A	B	C	D
Print time	4	7	2	5
Bind time	5	3	8	6

Johnson's Rule (Optimal for 2 machines!)

1. Find smallest time across both machines
2. On Machine 1 → schedule job *first*
3. On Machine 2 → schedule job *last*
4. Remove job, repeat

Step 1: C's print (2) → M1 → C first
Step 2: B's bind (3) → M2 → B last
Step 3: A's print (4) → M1 → A next
Step 4: D fills remaining

Sequence: C – A – D – B

Result

Printer:
C(0–2), A(2–6), D(6–11), B(11–18)

Binder:
C(2–10), A(10–15), D(15–21), B(21–24)

$C_{\max} = 24$

Compare random A–B–C–D: $C_{\max} = 27$

Saves 3 units (11%)

The Gap: When Rules Aren't Enough

Dispatching Rules

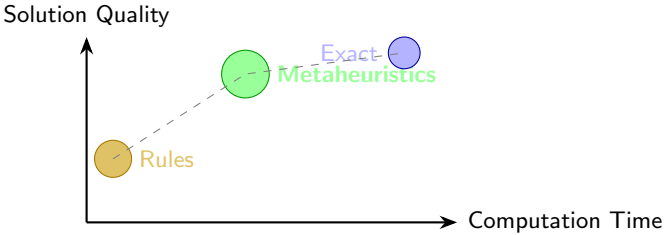
- ✓ Fast (ms)
- ✓ Easy to implement
- ✓ Good for real-time
- × Myopic
- × Often 10–30% from optimal

Exact Methods

- ✓ Provably optimal
- ✓ Handle any constraint
- × Exponential worst-case
- × May not finish for $n > 50$

Metaheuristics

- ✓ Near-optimal (1–5%)
- ✓ Reasonable time
- ✓ Handle complexity
- ✓ Flexible



What Is a Metaheuristic?

Meta (Greek) = "beyond" **Heuristic** (Greek) = "to find"

A **metaheuristic** is a high-level strategy that guides a search process to find good solutions to hard optimization problems.

Key Properties

- Problem-independent framework
- Uses randomness to escape local optima
- No guarantee of global optimum
- Balances *exploration* vs. *exploitation*

The Mountain Analogy

Lost in mountains at night, finding the lowest valley:

- **Exact:** Map every inch. Perfect but slow.
- **Rule:** Always walk downhill. Stuck in first dip.
- **Metaheuristic:** Mostly downhill, but occasionally climb a ridge to find a deeper valley.

Families of Metaheuristics

Trajectory-Based

Inspiration: Physics, metallurgy
Examples: Simulated Annealing, Tabu Search
Idea: Improve a *single* solution step by step

Population-Based

Inspiration: Biology, evolution
Examples: Genetic Algorithms, Particle Swarm
Idea: Evolve a *group* of solutions together

Constructive

Inspiration: Greedy strategies
Examples: GRASP, Ant Colony
Idea: Build solutions piece by piece

Today's Focus

- **Simulated Annealing** (trajectory)
- **Genetic Algorithms** (population)

The two most widely used "starter" metaheuristics.

Coming Later in the Course

- Tabu Search
- Ant Colony Optimization
- Particle Swarm
- Hybrid methods

Exploration vs. Exploitation

The central tension in *every* metaheuristic:

Exploration (Diversification)

- Search **broadly** across solution space
- Visit new, uncharted regions
- Accept worse solutions to escape traps
- Important **early** in the search

Analogy: Trying many different restaurants in a new city.

Exploitation (Intensification)

- Search **deeply** near known good solutions
- Refine and improve promising areas
- Only accept improvements
- Important **late** in the search

Analogy: Trying every dish at your favorite restaurant.

Too much exploration = wander forever, never converging. **Too much exploitation** = get stuck in a local optimum. Good metaheuristics **balance** both.

Core Concept: Neighborhoods and Moves

Neighborhood: The set of solutions reachable by making one small change (a "move").

For a schedule [A, B, C, D, E], common moves:

Swap

Exchange two positions.

[A, **B**, C, **D**, E]

↓

[A, **D**, C, **B**, E]

Size: $\binom{n}{2}$

n=5: 10 neighbors

Insert

Remove and reinsert.

[A, B, **C**, D, E]

↓

[A, B, D, E, **C**]

Size: $n(n-1)$

n=5: 20 neighbors

Reverse (2-opt)

Reverse a subsequence.

[A, **B, C, D**, E]

↓

[A, **D, C, B**, E]

Size: $\binom{n}{2}$

n=5: 10 neighbors

Your current schedule is a point on a map. The neighborhood is all the nearby points you could step to.

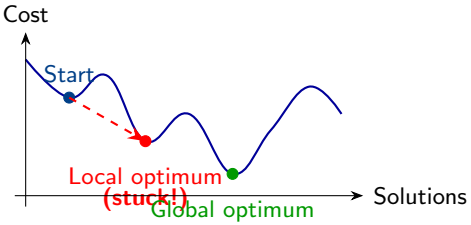
Hill Climbing: The Simplest Local Search

Algorithm:

1. Start with any solution
2. Look at all neighbors
3. Move to the best improving neighbor
4. Repeat until no neighbor improves
5. You're at a **local optimum**

The Problem

Hill climbing gets **stuck** in local optima! It always goes downhill (for minimization), so it can never escape a valley to find a deeper one.



We need a way to occasionally accept *worse* solutions to escape.

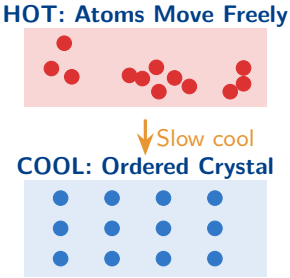
Enter: Simulated Annealing!

Simulated Annealing: Inspired by Metallurgy

What is annealing in materials science?

Annealing is a heat treatment used by metallurgists for thousands of years:

1. **Heat** metal to high temperature (atoms move freely, material is soft)
2. **Slow cooling** allows atoms to settle into a low-energy crystal structure
3. **Result:** fewer defects, stronger material



The analogy:

Hot → Accept many bad moves (explore)
Cold → Accept only good moves (exploit)

Why Slow Cooling Matters

Fast cooling (quenching) → atoms trapped in disorder → brittle metal.
Slow cooling (annealing) → atoms find optimal positions → strong metal.

Physical Annealing: The Science

Atomic Behavior

At temperature T, atoms have energy $\sim k_B T$.

- **High T:** Atoms vibrate wildly, rearrange freely
- **Medium T:** Rearrange but prefer low-energy states
- **Low T:** Locked in place, tiny adjustments only

Boltzmann Distribution

Probability of state with energy E at temperature T:

$$P(E) \propto e^{-E/(k_B T)}$$

Higher energy states possible but less likely at low T.

Real-World Annealing

- **Steel:** 700–900°C, cooled over hours to remove stresses
- **Glass:** Oven slowly cools from 500°C to prevent cracking
- **Silicon wafers:** 1000°C to repair crystal damage
- **Copper wire:** Annealed to restore ductility

Key insight: The same physics that helps atoms find low-energy states helps our algorithm find low-cost schedules!

Why Accept Worse Solutions?

This is the hardest idea for students. Let's build intuition.

Without Accepting Worse

You're a hiker looking for the lowest point. You always walk downhill. You reach a valley floor and stop.

But is it the lowest valley? You have no idea — the deepest valley might be just over the next ridge.

With Accepting Worse (SA)

Same hiker, but willing to climb a ridge *occasionally*:

- A deeper valley on the other side
- A shortcut to a much better region
- Nothing better (but at least you checked!)

As time goes on, you climb ridges less and less, eventually settling in the best valley found.

Accepting worse solutions is not random searching. It's a controlled strategy: accept worse moves with decreasing probability as the search progresses.

Simulated Annealing: The Algorithm

SA Pseudocode

```
s ← initial solution (e.g., SPT rule)
T ← T_0                                   (high starting temp)
while T > T_min do
    s' ← random neighbor of s
    Δ = f(s') − f(s)                      (cost change)
    if Δ ≤ 0 then
        s ← s'                            (always accept)
    else
        Accept s' with prob e^(−Δ/T)
    end if
    T ← α · T                             (cool down)
end while
return best s found
```

The Acceptance Probability

$P(\text{accept worse}) = e^{-\Delta/T}$

Δ	T	P
5	100	95.1%
5	10	60.7%
5	1	0.7%
50	100	60.7%
50	10	0.7%

Key Insight

High T: accept almost anything. Low T: accept only improvements. Small Δ: more likely accepted. Large Δ: less likely accepted.

SA Parameters: The Cooling Schedule

Three Parameters to Set

1. **Initial temperature T_0:**
   High enough that ~80% of worse moves are accepted. Rule of thumb: $e^{-\Delta_{\text{avg}}/T_0} \approx 0.8$.
2. **Cooling rate α:**
   Typically 0.90–0.99. Higher = slower = better quality but more time.
3. **Stopping criterion:**
   $T < T_{\min}$, or no improvement for k iterations, or time limit.

Geometric Cooling (Most Common)

$T_{k+1} = \alpha \cdot T_k$

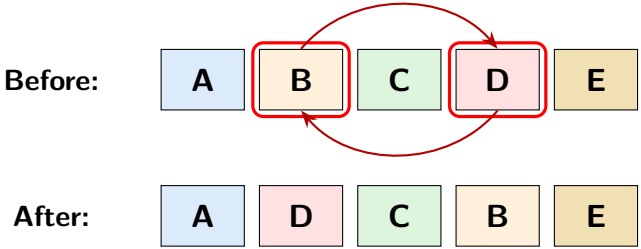
Example: $T_0 = 100$, $\alpha = 0.95$

Iteration	Temperature
0	100.0
10	59.9
20	35.8
50	7.7
100	0.6

Remember: Just like real annealing — cool too fast and you get a brittle, suboptimal result!

Visualizing the "Shuffle": Swap Move

Swap: Pick two positions and exchange the jobs.

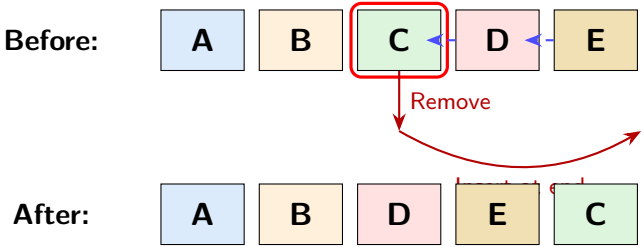


How: Choose 2 random positions (here 2 and 4). Swap the jobs.

Properties: $\binom{n}{2}$ possible swaps. For n=5: 10 neighbors. Gentle move.

Visualizing the "Shuffle": Insert Move

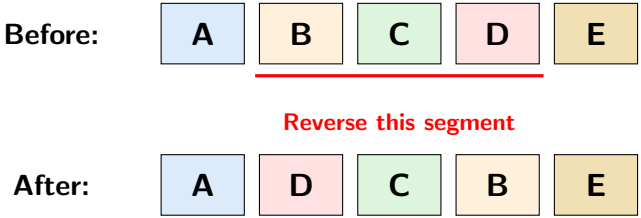
Insert: Remove a job and re-insert it at a different position.



Properties: $n(n-1)$ possible inserts. For $n=5$: 20 neighbors. More disruptive than swap.

Visualizing the "Shuffle": Reverse (2-opt)

Reverse: Pick a subsequence and reverse its order.



Classic "2-opt" move from TSP. $\binom{n}{2}$ possible reverses. Very effective for setup-cost problems.

SA can use any of these! The choice affects search quality. Experiment to find what works for your problem.

SA Example: Flow Shop Setup

Problem: 4 jobs, 2 machines (flow shop). Minimize makespan $C_{\max}$.

Job	Machine 1	Machine 2
J1	4	3
J2	2	5
J3	6	2
J4	3	4

SA Configuration

- **Encoding:** permutation, e.g., [J2, J1, J4, J3]
- **Neighbor:** swap two adjacent jobs
- **Parameters:** $T_0 = 50$, $\alpha = 0.8$, $T_{\min} = 1$

Initial Solution (random)

$s_0 =$ [J1, J2, J3, J4]
M1: J1(0–4), J2(4–6), J3(6–12), J4(12–15)
M2: J1(4–7), J2(7–12), J3(12–14), J4(15–19)
$C_{\max} = 19$

SA Example: How to Evaluate a Neighbor

Current: [J1, J2, J3, J4] with $C_{\max} = 19$. **Neighbor:** swap J1 and J2 $\rightarrow$ [J2, J1, J3, J4].

Compute New Makespan

Machine 1:
J2: start 0, end 2
J1: start 2, end 6
J3: start 6, end 12
J4: start 12, end 15

Machine 2: (must wait for M1)
J2: start max(2, 0) = 2, end 7
J1: start max(6, 7) = 7, end 10
J3: start max(12, 10) = 12, end 14
J4: start max(15, 14) = 15, end 19

$C_{\max} = 19$... same! But wait...

Try Another Neighbor

Swap J2 and J3: [J1, J3, J2, J4]

M1: J1(0–4), J3(4–10), J2(10–12), J4(12–15)
M2: J1(4–7), J3(10–12), J2(12–17), J4(17–21)

$C_{\max} = 21$. That's *worse* by 2!

$\Delta = 21 - 19 = +2$
$P(\text{accept}) = e^{-2/50} = 0.96$

At high temperature, we'd accept this 96% of the time!

Each iteration: (1) pick random neighbor, (2) compute Δ, (3) accept/reject, (4) cool down.

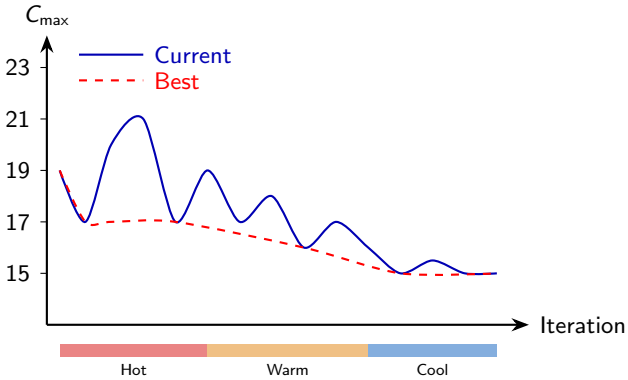
SA Example: Step-by-Step Iterations

Current: $s = [\text{J1, J2, J3, J4}]$, $f(s) = 19$, $T = 50$

Iter	**Neighbor** s'	$f(s')$	Δ	$e^{-\Delta/T}$	**Action**
1	[J2, J1, J3, J4] $s \leftarrow$ *[J2, J1, J3, J4]*, $f = 18$	18	-1	—	Accept (improvement)
2	[J2, J3, J1, J4] $s \leftarrow$ *[J2, J3, J1, J4]*, $f = 20$, $T \rightarrow 40$	20	$+2$	0.96	Accept (rand < 0.96)
3	[J2, J3, J4, J1] $s \leftarrow$ *[J2, J3, J4, J1]*, $f = 21$, $T \rightarrow 32$	21	$+1$	0.98	Accept (rand < 0.98)
4	[J2, J4, J3, J1] $s \leftarrow$ *[J2, J4, J3, J1]*, $f = 17$	17	-4	—	Accept (big improvement!)

Key: Iterations 2–3 accepted *worse* solutions but iteration 4 found a *much better* one! Without those uphill moves, we'd be stuck at 18. Best so far: $C_{\max} = 17$.

SA: How It Converges



Three Phases

1. **Hot:** Wild exploration. Cost jumps around. Many bad moves accepted.
2. **Warm:** Focused search. Mostly improving. Occasional bad move.
3. **Cool:** Fine-tuning. Almost pure hill climbing. Settles to good solution.

Best Practice

Always track the *best solution found* (red dashed), not just the current one.

SA Summary

Strengths

- Simple to implement (20–30 lines)
- Works on almost any problem
- Proven convergence (theory)
- Few parameters to tune
- Good at escaping local optima

Limitations

- Single-solution (no population)
- Sensitive to cooling schedule
- Can be slow for very large problems
- No memory of visited solutions

When to Use SA

You need a quick, general-purpose optimizer; the problem has a natural neighborhood; you want a good baseline without designing a problem-specific algorithm.

Genetic Algorithms: Inspired by Evolution

Darwin's Natural Selection:

1. A **population** of organisms exists
2. **Fitter** individuals survive and reproduce
3. Offspring inherit traits from **two parents**
4. Random **mutations** introduce variation
5. Over generations, population **evolves**

The Analogy

Biology	Optimization
Organism	Solution (schedule)
Chromosome	Encoding (permutation)
Fitness	Objective value
Population	Set of solutions

GA for Scheduling

1. Create initial population of random schedules
2. Evaluate each schedule's makespan
3. Select the best as "parents"
4. Combine parents $\rightarrow$ "children"
5. Randomly mutate some children
6. Replace worst with children
7. Repeat for many generations

Key Difference from SA

SA improves *one* solution at a time. GA improves a *population* simultaneously. Multiple search paths = better exploration.

GA Step 1: Encoding Schedules as Chromosomes

Permutation encoding is most natural for scheduling.
Each chromosome is a job sequence: **[J3, J1, J4, J2, J5]**

A Population of 6 Schedules

#	Chromosome	C_{max}
1	[J3, J1, J4, J2, J5]	24
2	[J1, J2, J3, J4, J5]	28
3	[J5, J4, J2, J1, J3]	22
4	[J2, J3, J1, J5, J4]	26
5	[J4, J5, J3, J2, J1]	23
6	[J1, J5, J4, J3, J2]	27

Why a Population?

- Each schedule explores a *different* region
- Good "building blocks" (subsequences) might be in different chromosomes
- Crossover can *combine* the best parts
- Diversity prevents getting stuck

GA Step 2: Selection — Who Gets to Reproduce?

Tournament Selection

1. Pick 2 random chromosomes
2. Better fitness wins
3. Winner becomes a parent
4. Repeat for second parent

Example:
Round 1: #3 ($C_{\max}$=22) vs #4 (26) → #3 wins
Round 2: #5 ($C_{\max}$=23) vs #2 (28) → #5 wins

Parents: #3 and #5

Why Tournament?

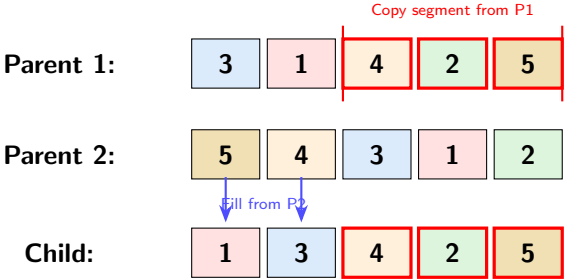
- Simple to implement
- Adjustable pressure: larger tournament = stronger selection
- Doesn't require sorting the whole population
- Better individuals have *higher* chance but not guaranteed — maintains diversity

Other Methods

Roulette wheel, rank-based, truncation. Tournament is most common for scheduling.

GA Step 3: Crossover — Combining Parents

Order Crossover (OX) preserves relative ordering:



Steps: (1) Choose cut points 3–5. (2) Copy [4,2,5] from P1. (3) From P2 order (5,4,3,1,2), remaining: 3,1. (4) Fill: [1, 3, 4, 2, 5].

Why OX? Normal crossover creates invalid schedules (duplicate jobs). OX guarantees each job appears exactly once.

Understanding OX Crossover: Step by Step

Let's slow down and trace OX carefully.

Given

P1: [3, 1, | **4, 2, 5** |]
P2: [5, 4, 3, 1, 2]
Cut points: after position 2 and after position 5

Step 1: Copy segment from P1

Child = [_, _, **4, 2, 5**]
Positions 3–5 are filled.

Step 2: List P2's jobs in order

Starting after the second cut point in P2:
P2 order: 5, 4, 3, 1, 2
(wrap around from the cut point)

Step 3: Remove already-placed jobs

Already placed: {4, 2, 5}
From P2 order [5, 4, 3, 1, 2]:
Remove 5 → [4, 3, 1, 2]
Remove 4 → [3, 1, 2]
Remove 2 → [**3, 1**]
Remaining jobs in order: 3, 1

Step 4: Fill empty positions

Fill from left:
Position 1 ← 3
Position 2 ← 1

Child = [3, 1, 4, 2, 5]
Wait, that's P1 itself! Different cut points or parents would produce a truly new child.

GA Step 4: Mutation — Adding Randomness

Mutation prevents the population from becoming too similar.

Swap Mutation

Pick two random positions and swap:
[J1, J3, **J4**, J2, **J5**]
↓
[J1, J3, **J5**, J2, **J4**]
Mutation probability: typically 5–20%.

Inversion Mutation

Reverse a random segment:
[J1, **J3, J4, J2**, J5]
↓
[J1, **J2, J4, J3**, J5]

Why Mutation Matters

Without mutation:

- Population converges too quickly
- All chromosomes become similar
- Search stalls at local optimum

With mutation:

- New genetic material introduced
- Diversity maintained
- Can discover new regions

Analogy: Most mutations are neutral or harmful, but selection filters the bad ones and rare beneficial mutations drive evolution.

GA: The Complete Algorithm

GA Pseudocode

```
P ← random population of N schedules
Evaluate fitness of each
for gen = 1 to G do
    P' ← ∅
    while |P'| < N do
        Select parents p1, p2 (tournament)
        c1, c2 ← Crossover(p1, p2)
        Mutate c1, c2 with prob pm
        Add c1, c2 to P'
    end while
    Evaluate fitness of P'
    P ← best N from P ∪ P'
end for
return best schedule in P
```

Typical Parameters

Parameter	Range
Population N	50–200
Generations G	100–1000
Crossover rate	0.7–0.9
Mutation rate p_m	0.05–0.2
Tournament size	2–5

Elitism

Always keep the *best* solution from the current generation. Prevents losing the best found so far.

GA Example: Paint Shop (from Week 3)

Recall: 4 colors, sequence-dependent setup times.

Setup time matrix (minutes):

From\To	R	B	G	Y
Start	5	5	5	5
Red	0	8	6	4
Blue	8	0	3	7
Green	6	3	0	5
Yellow	4	7	5	0

Processing: R=10, B=12, G=8, Y=15

Goal: Minimize total setup time

Initial Population ($N = 4$)

#	Chromosome	Setup
1	[R, B, G, Y]	21
2	[Y, R, G, B]	17
3	[G, B, Y, R]	19
4	[B, G, R, Y]	18

Selection

Tournament: #1 (21) vs #2 (17) → #2

Tournament: #3 (19) vs #4 (18) → #4

GA Example: Crossover and Mutation

Parents: #2 = [Y, R, G, B] and #4 = [B, G, R, Y]

Order Crossover (cut pos 2–3)

P1: [Y, | **R, G**, | B]
P2: [B, G, R, Y]

Child 1: Copy R, G from P1.
Remaining from P2 order: B, Y
Result: [**B**, R, G, **Y**]
Setup $= 5 + 8 + 6 + 5 = 24$

Child 2: [R, G, B, Y]
Setup $= 5 + 6 + 3 + 5 = 19$

Mutation on Child 1

[B, R, G, Y] $\rightarrow$ swap pos 1 and 2:
R, B, G, Y
$\rightarrow$ setup $= 5 + 8 + 3 + 5 = 21$

New Population (keep best 4)

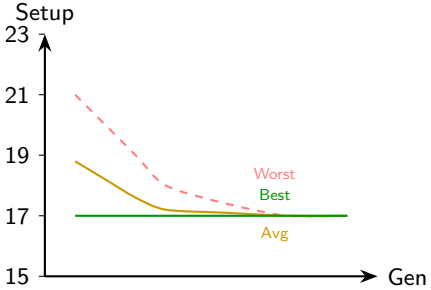
#	Schedule	Setup
2	[Y, R, G, B]	**17**
4	[B, G, R, Y]	18
C2	[R, G, B, Y]	19
C1m	[R, B, G, Y]	21

Best is still 17. GA continues...

GA Example: Evolution Over Generations

Gen	Best	Avg	Worst	Best Schedule
1	17	18.8	21	[Y,R,G,B]
2	17	18.2	20	[Y,R,G,B]
3	17	17.5	18	[Y,R,G,B]
5	17	17.2	18	[B,G,Y,R]
10	17	17.0	17	[Y,R,G,B]

After 10 generations, the whole population converged to setup $= 17$ (this is optimal).



Population converges: worst and average approach the best.

Multiple Optima Found

[Y, R, G, B]: $5 + 4 + 6 + 3 = 17$
[B, G, Y, R]: $5 + 3 + 5 + 4 = 17$
Same as Nearest Neighbor from Week 3!

GA Summary

Strengths

- Population explores many regions
- Crossover combines good building blocks
- Naturally parallelizable
- Effective for complex search spaces

Limitations

- More parameters than SA
- Crossover design is problem-specific
- Can prematurely converge
- Heavier computation per iteration

SA vs. GA: When to Use Which?

Criterion	Choose SA	Choose GA
Problem knowledge	Limited	Can design good crossover
Time budget	Small (fast)	Larger (better quality)
Landscape	Many local optima	Structured, decomposable
Parallelism	Not needed	Available

A Realistic Job Shop Scenario

Scenario: You manage a job shop with 8 machines and 50 jobs. New jobs arrive daily. Schedule needed by 6 AM.

Your Toolkit

1. **Dispatching rule** (SPT/EDD) for initial schedule — 1 second
2. **SA** to improve — 30 seconds
3. **GA** for overnight optimization — 10 min
4. **MILP solver** for small subproblems

Typical Workflow

1. 5:00 AM: New data arrives
2. 5:01 AM: SPT schedule (baseline)
3. 5:02 AM: SA for 5 min ($\sim$10% better)
4. 5:07 AM: GA for 50 min ($\sim$3% more)
5. 5:57 AM: Final schedule ready
6. 6:00 AM: Shop floor starts

Real-world scheduling is layered. Use fast methods first, then refine. Don't choose one tool — use them together!

Method Comparison at a Glance

Method	Speed	Quality	Ease	Scale	Theory
FIFO/SPT/EDD	★★★★★	★★	★★★★★	★★★★★	★★★
MILP (exact)	★	★★★★★	★★	★	★★★★★
Sim. Annealing	★★★	★★★★	★★★★	★★★★	★★★
Genetic Alg.	★★	★★★★	★★★	★★★★	★★★

Key Takeaway

No single "best" method. The right choice depends on problem size, time budget, quality needs, and your expertise.

Industry Trend

Modern systems use **hybrid approaches**: rules for real-time, metaheuristics for medium-term, exact for strategic planning.

Common Pitfalls

1: Bad Initial Solution

Random start wastes iterations. **Fix:** Initialize with a dispatching rule.

2: Wrong Neighborhood

Too small = slow. Too large = random. **Fix:** Try multiple move types.

3: No Termination Strategy

Running forever doesn't help. **Fix:** Stop if no improvement for k iterations.

4: Premature Convergence (GA)

Population becomes identical. **Fix:** Increase mutation, diverse init.

5: Cooling Too Fast (SA)

Like quenching metal. **Fix:** Use $\alpha \geq 0.95$ for large problems.

6: Not Tracking Best

SA's current may worsen. **Fix:** Save best found separately.

Debugging Your Metaheuristic

How to tell if your metaheuristic is working:

Plot the Convergence Curve

- Plot best-so-far vs. iteration
- **Healthy:** rapid improvement early, gradual later
- **Problem:** flat from the start $\rightarrow$ too cold or bad neighborhood
- **Problem:** never settles $\rightarrow$ too hot or cooling too slow

Check Acceptance Rate (SA)

- Track % of moves accepted per 100 iterations
- **Start:** should be $\sim$80%
- **End:** should be $<$5%
- If always high $\rightarrow$ T_0 too high
- If always low $\rightarrow$ T_0 too low

For GA: Check Population Diversity

If all chromosomes become identical before converging to a good solution, increase mutation rate or use a diversity-preserving selection method.

Practice Problem 1: Apply SA by Hand

Problem: 5 jobs on 1 machine. Minimize total tardiness $\sum T_j$.

Job	A	B	C	D	E
Processing time	4	6	2	8	3
Due date	8	12	5	20	10

Your Task

1. Start with EDD sequence
2. Compute total tardiness
3. Generate one swap neighbor
4. Compute Δ and acceptance probability at $T = 20$
5. Accept or reject?

Hint: EDD Order

Sort by due date:
C(5), A(8), E(10), B(12), D(20)

Completions:
C: 2, A: 6, E: 9, B: 15, D: 23

Tardiness: 0, 0, 0, 3, 3 = **6**

Practice Problem 1: Solution Walkthrough

Starting: [C, A, E, B, D] with $\sum T_j = 6$.
Generate neighbor: Swap B and E $\rightarrow$ [C, A, B, E, D]

Evaluate Neighbor

Job	Start	End	Due	T_j
C	0	2	5	0
A	2	6	8	0
B	6	12	12	0
E	12	15	10	5
D	15	23	20	3

$\sum T_j = 8$

SA Decision

$\Delta = 8 - 6 = +2$ (worse!)
$P(\text{accept}) = e^{-2/20} = e^{-0.1} = 0.905$

At $T = 20$, we'd accept this worse solution **90.5% of the time**.

Generate random number $r \in [0, 1]$.
If $r < 0.905 \rightarrow$ accept.
If $r \geq 0.905 \rightarrow$ reject.

Practice Problem 2: GA Crossover by Hand

Perform Order Crossover (OX):
Parent 1: [D, A, E, B, C] Parent 2: [C, B, A, D, E]
Cut points: positions 2 and 4 (inclusive)

Child 1 (from P1 segment)

1. Copy pos 2–4 from P1: [_, **A, E, B**, _]
2. P2 order: C, B, A, D, E
3. Remove A, E, B: remaining = C, D
4. Fill: [**C**, A, E, B, **D**]

Child 2 (from P2 segment)

1. Copy pos 2–4 from P2: [_, **B, A, D**, _]
2. P1 order: D, A, E, B, C
3. Remove B, A, D: remaining = E, C
4. Fill: [**E**, B, A, D, **C**]

Try these problems before next class. We'll discuss at the start of next session.

Key Formulas Reference Card

SA Acceptance Probability

$$P(\text{accept}) = \begin{cases} 1 & \text{if } \Delta \leq 0 \\ e^{-\Delta/T} & \text{if } \Delta > 0 \end{cases}$$

where $\Delta = f(s') - f(s)$

Geometric Cooling

$$T_{k+1} = \alpha \cdot T_k, \quad 0.90 \leq \alpha \leq 0.99$$

Combinatorial Explosion

Single machine: $n!$ Parallel: $\sim (n!)^m$

Dispatching Rules

SPT: Sort by p_j ascending
EDD: Sort by d_j ascending
WSPT: Sort by w_j/p_j descending
CR: Priority $= (d_j - t)/p_j$
LPT: Sort by p_j descending

Tardiness

$$T_j = \max(0,\ C_j - d_j)$$

Neighborhood Sizes

Swap: $\binom{n}{2}$ Insert: $n(n-1)$ Reverse: $\binom{n}{2}$

What's Next: Beyond SA and GA

Tabu Search

Like hill climbing, but keeps a "tabu list" of recent moves to prevent cycling. Memory-based search.

Particle Swarm Optimization

A "swarm" of particles flies through solution space. Each remembers its best and is attracted to the global best.

Ant Colony Optimization

Artificial ants deposit "pheromones" on good paths. Best paths get stronger signals. Great for routing.

Hybrid Methods

GA for global search + SA for local refinement = powerful synergy. Modern state of the art.

Master SA and GA first — they're the foundation for everything else.

Real-World Impact of Scheduling Optimization

Semiconductor Fab

GA-based scheduling reduced cycle time by 15% and increased throughput by 8% in a 300mm wafer fab.

Airline Crew Scheduling

SA-based crew scheduling saves major airlines \$50–100M/year by reducing deadheading and overtime.

Automotive Paint Shop

TSP-based color sequencing (SA) reduced paint waste by 20% and setup time by 30%.

Hospital OR Scheduling

Metaheuristics increased OR utilization from 68% to 82%, reducing wait times by 25%.

The Bottom Line

Companies using advanced scheduling report:

- 10–25% reduction in makespan
- 15–30% reduction in WIP
- 20–40% reduction in tardiness
- 5–15% increase in throughput

References

- Pinedo, M.L. (2016). *Scheduling: Theory, Algorithms, and Systems*, 5th ed. Springer.
- Kirkpatrick, S., Gelatt, C.D., & Vecchi, M.P. (1983). "Optimization by Simulated Annealing." *Science*, 220(4598), 671–680.
- Holland, J.H. (1975). *Adaptation in Natural and Artificial Systems*. U. Michigan Press.
- Goldberg, D.E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
- Sodhi, M.S., Son, B.-G., & Tang, C.S. (2012). "Researchers' Perspectives on Supply Chain Risk Management." *Prod. and Ops. Mgmt.*, 21(1), 1–13.
- Sodhi, M. & Tang, C. (2009). "Modeling Supply-Chain Planning Under Demand Uncertainty Using Stochastic Programming." *Int. J. Prod. Econ.*, 121(2), 728–738.
- Williams, H.P. (2013). *Model Building in Mathematical Programming*, 5th ed. Wiley.
- Winston, W.L. (2004). *Operations Research*, 4th ed. Thomson.

Additional Resources

Software Tools

- **Python:** PuLP, Pyomo, OR-Tools
- **SA:** `simanneal` package
- **GA:** DEAP, PyGAD
- **Solvers:** Gurobi, CPLEX (free academic)

Benchmarks

- NEOS Server: free optimization
- OR-Library: standard instances
- Taillard's benchmarks for flow/job shop

Key Papers on GAs for Scheduling

- Davis, L. (1985). "Job-Shop Scheduling with Genetic Algorithms." *Proc. ICGA.*
- Bean, J.C. (1994). "Genetic Algorithms and Random Keys for Sequencing." *ORSA J. Computing.*
- Gen, M. & Cheng, R. (2000). *GA and Engineering Optimization.* Wiley.
- Sodhi et al. — for GA applications to supply chain and production scheduling.

Week 6 Preview: Lot Sizing, MRP & Production Planning

Lot Sizing Optimization

- **EOQ model:** The Costco vs. corner store trade-off — how much to order?
- **EPQ model:** When production rate is finite
- **Wagner-Whitin:** Optimal dynamic lot sizing via DP
- **Silver-Meal heuristic:** Fast approximation
- Quantity discounts and sensitivity analysis

Material Requirements Planning

- Exploding Bills of Materials (BOM)
- Dependent demand calculations
- MRP logic: gross $\rightarrow$ net $\rightarrow$ planned orders
- Lead time offsetting
- Lot sizing within MRP (L4L, EOQ, Silver-Meal)
- MRP nervousness and mitigation

Key Connection

Week 5: *how* to solve scheduling problems.
Week 6: *what* to schedule (lot sizes and quantities).

Key Takeaways

The Big Ideas

1. **Scheduling is hard:** NP-hard means we can't brute-force real problems
2. **Dispatching rules** are fast but myopic — good for real-time
3. **Metaheuristics** fill the gap between rules and exact methods
4. **SA** mimics physical annealing — slowly cool from exploration to exploitation
5. **GA** mimics evolution — populations evolve toward fitness

Skills Gained

- Apply SPT, EDD, LPT rules by hand
- Trace SA iterations step-by-step
- Perform GA crossover and mutation
- Choose the right method for a problem
- Recognize when exact methods won't work

Remember

"A good solution now is better than a perfect solution never."

— applies to optimization too

Questions?

ISE 573: Manufacturing Execution Systems

Week 5: Scheduling Optimization & Metaheuristics

Department of Industrial Engineering — University of Rhode Island

"Go forth and metaheuristically optimize!"

Contact: `sodhi@uri.edu`